دانشگاه شهید بهشتی
دانشکده مهندسی و علوم کامپیوتر
آزمایشگاه رباتیک

برنامه کارآموزی آزمایشگاه رباتیک

تمرین هفته پنجم: تکمیل و پایدارسازی سیستم اجرای Task

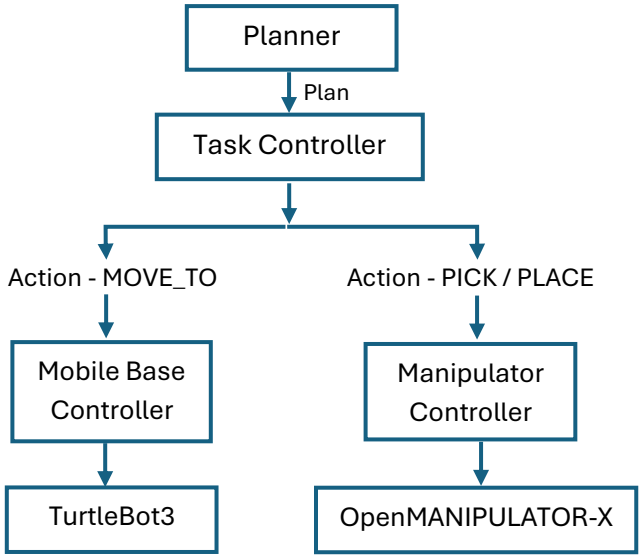
مقدمه

در تمرین‌های چهار هفته گذشته، سیستم به‌تدریج از یک Task Controller ساده به یک سیستم رباتیک شامل TurtleBot3 و OpenMANIPULATOR-X توسعه پیدا کرد. در این مرحله، هدف اصلی اضافه کردن یک بخش کاملاً جدید به سیستم نیست، بلکه باید معماری و قابلیت‌های فعلی را کامل‌تر، قابل اطمینان‌تر و قابل استفاده‌تر کنید.

در این هفته، تمرکز اصلی روی Task Execution، مدیریت وضعیت، Failure Handling، استفاده صحیح از ROS 2 Actions و تست سیستم خواهد بود. سیستم نهایی باید بتواند یک Plan تولیدشده توسط PDDLStream را دریافت کرده، مراحل آن را به ترتیب اجرا کند، وضعیت اجرای هر مرحله را مشخص کند و در صورت بروز خطا، اجرای Plan را به شکل کنترل‌شده متوقف کند.

هدف پروژه:

توجه داشته باشید که ممکن است برخی از قابلیت‌های بیان‌شده در تمرین این هفته را هفته‌های گذشته پیاده‌سازی کرده باشید. در این صورت نیازی به پیاده‌سازی دوباره نیست؛ هدف حاصل کردن اطمینان از این است که در پایان این تمرین، سیستم از حالت یک Pipeline ساده که فقط Actionها را یکی پس از دیگری اجرا می‌کند، به یک سیستم Task Execution قابل کنترل‌تر تبدیل شود .به‌صورت کلی، ساختار مورد انتظار به شکل زیر است:



در این تمرین لازم نیست Planner جدیدی توسعه دهید و نباید PDDLStream را تغییر دهید. همچنین نیازی به اضافه کردن Perception یا بخش بزرگی مانند Computer Vision نیست، تمرکز پروژه روی بهبود بخش Execution سیستم موجود است.

تبدیل ارتباط اجرای Task به ROS 2 Action:

در معماری فعلی، اجرای بعضی عملیات‌ها با Topic انجام می‌شود و نتیجه آن‌ها نیز از طریق یک Topic جداگانه گزارش می‌شود.
برای مثال، در نسخه فعلی ارتباط‌هایی مانند:

```
/task/action
/action_done
```

برای اجرای بعضی عملیات‌ها استفاده شده‌اند. در این هفته باید این معماری را برای عملیات‌های اصلی اجرای Task به سمت ROS 2 Action منتقل کنید. حداقل بایدAction های مورد نیاز برای این دو بخش مشخص شوند:

```
Mobile Base
Manipulator
```

با ساختاری مشابه:

```
# Goal
float64 target_x
float64 target_y
float64 target_theta

---
# Result
bool success
string message

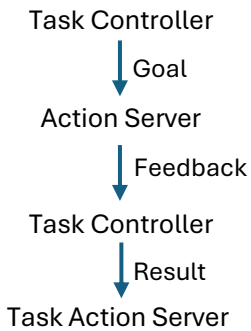
---
# Feedback
float64 distance_remaining
int32 current_waypoint
int32 total_waypoints
```

در صورت نیاز می‌توانید Interface را با توجه به ساختار واقعی trajectory و controller خود تغییر دهید. برای عملیات Manipulation نیز می‌توانید Action مناسب طراحی کنید.
برای مثال:

```
Manipulate.action
```

یا در صورت مناسب بودن ساختار فعلی سیستم، Actionهای مشخص‌تری برای عملیات‌هایی مانند Pick و Place تعریف کنید.
همچنین Action باید امکان Cancel کردن عملیات را داشته باشد.

هدف این بخش صرفاً ساختن یک فایل action. نیست. باید ارتباط کامل زیر برقرار باشد:



بازطراحی Task Controller

Task Controllerباید همچنان Plan تولیدشده توسط PDDLStream را دریافت و به ترتیب اجرا کند. اما در این مرحله Task Controller نباید فقط منتظر یک پیام action_done باشد. برای هر Action باید بتواند:

- Goal ارسال کند.
- Feedback و Result دریافت کند.
- موفقیت یا Failure عملیات را تشخیص دهد.
- در صورت موفقیت به Action بعدی برود و در صورت Failure اجرای Plan را متوقف کند.
- در صورت نیاز Action جاری را Cancel کند.
- وضعیت کلی Task را نگهداری کند.

مثلاً خروجی Task Controller می‌تواند چیزی شبیه این باشد:

```
TASK STARTED
[1/4] MOVE_BASE
  Waypoint: 3/18
  Distance remaining: 1.42 m
  SUCCESS

[2/4] PICK_AND_STOW
  APPROACH
  GRASP
  STOW
  SUCCESS

[3/4] MOVE_BASE
  Waypoint: 7/21
  Distance remaining: 0.83 m
  SUCCESS

[4/4] UNSTOW_AND_PLACE
  ...
  SUCCESS
TASK COMPLETED
```

در صورت Failure:

```
[3/4] MOVE_BASE
  FAILED
TASK FAILED
Reason: Robot did not reach waypoint
```

مدیریت وضعیت Task

Task Controller باید وضعیت کلی اجرای Plan را نگهداری کند. حداقل وضعیت‌های زیر را در نظر بگیرید:

- IDLE
- EXECUTING
- FAILED
- CANCELING
- CANCELED
- COMPLETED

لازم نیست حتماً دقیقاً از همین نام‌ها استفاده کنید، اما وضعیت‌های سیستم باید مشخص و قابل تشخیص باشند. در این بخش می‌توانید از مفاهیمی که در بخش آموزشی هفته پنجم درباره Parameters و Actions یاد گرفته‌اید استفاده کنید. همچنین مشخص باشد که:

- کدام Plan در حال اجرا است.
- کدام Action در حال اجرا است.
- چند Action از Plan انجام شده است.
- Action فعلی چیست.
- وضعیت Action فعلی چیست.
- Task در چه وضعیتی قرار دارد.

مدیریت Failure

در پروژه‌های قبلی، بعضی Failureها مانند Timeout، قطع شدن odom/، گیر کردن ربات، مسیر نامعتبر و Action ناشناخته در Robot Controller تشخیص داده می‌شدند. در این هفته باید بررسی کنید که این Failureها چگونه باید به لایه بالاتر یعنی Task Controller منتقل شوند. Task Controller نباید در صورت Failure یک Action، به‌صورت خودکار Action بعدی را اجرا کند. برای مثال اگر MOVE_BASE شکست خورد، نباید بدون بررسی وارد PICK شود. حداقل یک Failure واقعی را در سیستم ایجاد و رفتار سیستم را بررسی کنید.

Cancel کردن Task

یکی از مزیت‌های اصلی استفاده از Action این است که عملیات طولانی را می‌توان در هنگام اجرا Cancel کرد. پس از Cancel شدن Action، Task Controller باید وضعیت مناسب را ثبت کند و نباید Action بعدی را اجرا کند.

Emergency Stop منطقی

یک مکانیزم ساده برای توقف کل Task ایجاد کنید. این بخش یک Emergency Stop نرم‌افزاری برای شبیه‌سازی و کنترل منطقی سیستم است و نباید به عنوان Safety-rated Emergency Stop در نظر گرفته شود. می‌توانید یک Service، Topic یا Interface مناسب برای این کار انتخاب کنید.

در این حالت Action جاری باید Cancel شود، فرمان حرکتی جدید نباید ارسال شود، Action بعدی نباید اجرا شود و وضعیت Task باید مشخصاً به حالت توقف اضطراری منتقل شود. در صورت استفاده از سخت‌افزار واقعی، این قابلیت را صرفاً به عنوان کنترل نرم‌افزاری در نظر بگیرید و برای Safety واقعی از مکانیزم‌های ایمنی خود ربات استفاده کنید.

استفاده از Parameters برای تنظیمات سیستم

بخشی از تنظیمات سیستم که در حال حاضر مستقیماً داخل کد نوشته شده‌اند را به ROS 2 Parameter تبدیل کنید. لازم نیست همه موارد را تبدیل کنید، چند مورد مناسب انتخاب کنید. هدف این است که بتوان تنظیمات مهم را بدون تغییر کد تغییر داد. همچنین بررسی کنید که اگر مقدار نامعتبر برای یک Parameter وارد شود، Node چه رفتاری خواهد داشت.

Logging و وضعیت اجرای سیستم

خروجی Terminal برنامه باید برای Debugging قابل استفاده باشد. پیام‌های مهم باید به شکل مشخص ثبت شوند. از چاپ پیام‌های غیرضروری خودداری کنید و تلاش کنید پیام‌های مهم ساختار مشخصی داشته باشند. همچنین در صورت Failure، دلیل Failure باید تا حد امکان مشخص باشد.

تست سیستم

در این مرحله فقط اجرای موفق سیستم کافی نیست. حداقل چند Test برای رفتارهای اصلی سیستم اضافه کنید. هدف این بخش این است که رفتار سیستم فقط در حالت عادی بررسی نشود.

استفاده از rosbag2 برای ثبت یک اجرای سیستم

در پایان، حداقل یک اجرای کامل سیستم را با rosbag2 ضبط کنید. Topicهای مناسب را با توجه به معماری نهایی خود انتخاب کنید. سپس یک Task را اجرا کنید، داده‌های مربوط به اجرای آن را Record کنید،Bag را با ros2 bag info بررسی کنید، آن را Playback کنید و بررسی کنید آیا داده‌های مهم اجرای Task در Bag قابل مشاهده هستند یا خیر.

هدف این قسمت ایجاد یک روش ساده برای Debugging و بررسی اجرای سیستم است.

نکته پایانی

هدف این پروژه توسعه یک سیستم جدید نیست. باید تا حد امکان از Nodeها، Controllerها، Messageها و ساختار موجود در پروژه‌های هفته‌های قبل استفاده کنید و آن‌ها را تکمیل و تمیزتر کنید.

تمرکز اصلی روی این است که سیستم فعلی از یک Demo قابل اجرا به یک Task Execution System با مدیریت وضعیت، Failure، Recovery و Cancellation تبدیل شود.

